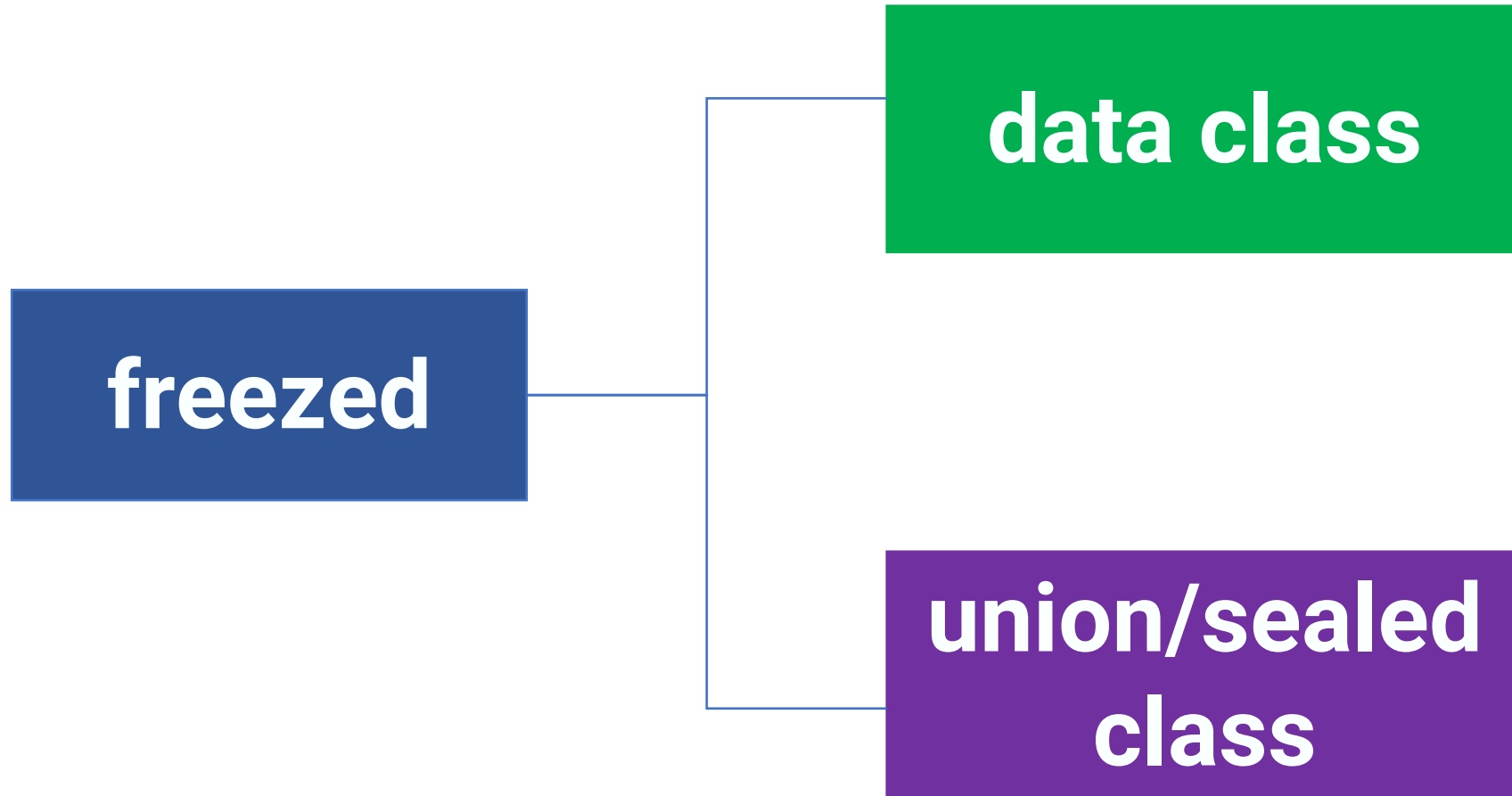




Dart 3

pattern matching

class modifiers



freezed package homepage

(Legacy) Union types and Sealed classes

Edit: As of Dart 3, Dart now has built-in pattern-matching using sealed classes.

As such, you no-longer need to rely on Freezed's generated methods for pattern matching. Instead of using `when / map`, use the official Dart syntax.

The following docs are left unedited and kept for the Dart users who have yet to migrate to Dart 3.

But in the long term, you should stop relying on `when / map`.

https://twitter.com/remi_rousselet/status/1673259042343452673



Remi Rousselet ✓

@remi_rousselet

Freezed users: Consider migrating your "when"/"map" usages to the Dart 3 pattern matching.

Since it is now an official Dart feature, we should use it :)

6:16 PM · Jun 26, 2023 · **20.1K** Views

https://twitter.com/remi_rousselet/status/1673377517548236800



Remi Rousselet ✓

@remi_rousselet

The next version of Freezed will disable the generation of when/map.

They will still be available by manually enabling them.

But disabling them should remove thousands of lines of generated code.

2:07 AM · Jun 27, 2023 · **7,109** Views

dart data class

- define properties and a constructor
- override `toString`, `operator ==`, `hashCode`
- implement a `copyWith` method to clone the object
- handing serialization/de-serialization

dart data class generator

- 💡 Generate data class
- 💡 Generate constructor
- 💡 Generate copyWith
- 💡 Generate JSON serialization
- 💡 Generate toString
- 💡 Generate Equatable

frozen

code generation

factory constructor can be unnamed

```
class MyClass {  
    factory MyClass() {  
        // Do something like returning an instance of MyClass  
    }  
}
```

```
class MyClass {  
    factory MyClass.fromJson(Map<String, dynamic> json) {  
        // Generate MyClass instance from json  
    }  
}
```

simplified syntax for constructor redirection

```
class Example {  
    factory Example({required String name, require int age}) {  
        return _Example(name: name, age: age);  
    }  
    void printName() {}  
    void printAge() {}  
}  
  
class _Example implements Example {  
    final String name;  
    final int age;  
    _Example({required this.name, required this.age});  
  
    @override  
    void printName() {}  
    void printAge() {}  
}
```


simplified syntax for constructor redirection

```
class Example {  
    factory Example({required String name, require int age}) = _Example;  
    void printName() {}  
    void printAge() {}  
}  
  
class _Example implements Example {  
    final String name;  
    final int age;  
    _Example({required this.name, required this.age});  
  
    @override  
    void printName() {}  
    void printAge() {}  
}
```

factory constructor can be declared as const

```
class Example {  
    const factory Example() = _Example;  
}  
  
class _Example implements Example {  
    const _Example();  
}
```

install

```
$ flutter(or dart) pub add freezed_annotation  
$ flutter(or dart) pub add --dev build_runner  
$ flutter(or dart) pub add --dev freezed
```

+

fromJson/toJson

```
$ flutter(or dart) pub add json_annotation  
$ flutter(or dart) pub add --dev json_serializable
```

annotation

**freezed_annotation
json_annotation**



code generator

build_runner

run

**freezed
json_serializable**

code

**.freezed.dart
.g.dart**

analysis_options.yaml

```
analyzer:  
  errors:  
    invalid_annotation_target: ignore  
  exclude:  
    - '**/*.freezed.dart'  
    - '**/*.g.dart'
```

running generator

```
$ dart run build_runner build [--delete-conflicting-outputs]  
$ dart run build_runner watch [-d]
```

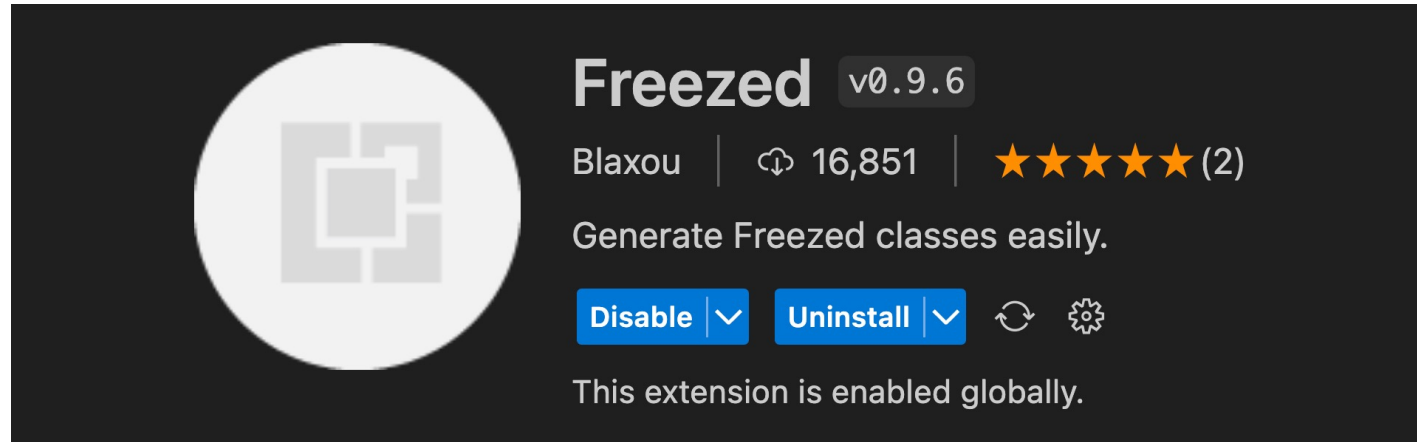
person.dart

```
import 'package:freezed_annotation/freezed_annotation.dart';
import 'package:flutter/foundation.dart';

part 'person.freezed.dart';
part 'person.g.dart';

@freezed
class Person with _$Person {
  const factory Person({
    required String firstName,
    required String lastName,
    required int age,
  }) = _Person;

  factory Person.fromJson(Map<String, Object?> json) => _$PersonFromJson(json);
}
```



ptf	⇒	part 'person.freezed.dart';
pts	⇒	part 'person.g.dart';
fdataclass	⇒	@freezed class DataClass with _\$DataClass{ const factory DataClass() = _DataClass; }
funion	⇒	@freezed class Union with _\$Union{ const factory Union.() = ; }